Taller de Programación I

Ferris

Levi, Dolores (105993)
La Rosa, Martina (97056)
Menechian, Javier (107942)
Benito, Agustin (108100)

1. Modelado de Datos 3
○ Diseño del esquema de la base de datos, incluyendo keyspaces y tablas. Definición de particiones y clustering de columnas para optimizar el rendimiento. 3
2. Operaciones CRUD 5
● Operaciones de creación, lectura, actualización y eliminación utilizando CQL (Cassandra Query Language) 5
○ Implementación de Read-Repair 6
3. Cliente - Servidor 7
4. Replicación y Consistencia 8
○ Decisiones de particionamiento 8
○ Configuración de estrategias de replicación (Replication Factor) 9
○ Consistent Hashing 10
○ Soporte a distintos niveles de consistencia (Consistency Level) 11
5. Protocolo Nativo de Cassandra 11
○ Tipos de mensajes 11
6. Funcionamiento de Clúster 12
○ Implementación del protocolo Gossip para comunicación entre nodos 12
○ Tolerancia a fallo de uno o más nodos del cluster 13
○ Implementación de Hinted Handoff para recuperación de datos ante la desconexión de un nodo 13
○ Punto Bonus (opcional): demostrar la compatibilidad utilizando un cliente comercial de Cassandra. Próximamente… 13
7. Seguridad 14
○ Autenticación y autorización 14
○ Encriptación de datos en tránsito (in-transit) 14
○ Encriptación de datos en reposo (at-rest) 14
8. Simulador de Vuelos 15
9. Interfaz gráfica 17
○ Módulos 17
○ Ver vuelos y editar el Estado de los vuelos 18
○ Integración del simulador 19
10. Links útiles 19
Final 2C 2024: Containerization 20

1. Modelado de Datos

- Diseño del esquema de la base de datos, incluyendo keyspaces y tablas.
  Definición de particiones y clustering de columnas para optimizar el rendimiento.

Nuestra implementación cuenta con 6 tipos de tabla diferentes a lo largo de la ejecución.
De estas, tenemos 3 tablas principales que harán uso de las operaciones CRUD mientras que las otras 2 simplemente brindan soporte a partes específicas del proyecto.

El porqué de la elección de estas tablas se verá en el punto 4. Aquí simplemente veremos las propiedades de cada tabla.

- **`origin`** table:

airport	flight	date	destination	status	timestamp

Tabla de estados de vuelos.

Sobre qué representa cada columna:
Airport: Aeropuerto de origen
Flight: Número de vuelo
Date: Fecha del vuelo
Destination: Aeropuerto de destino
Status: Estado del vuelo
Timestamp: Marca cuando se hizo la última actualización a la tabla

Partición: airport,date.
primary key: airport, date, flight.

- **`destination`** table:

airport	flight	date	origin	status	timestamp

Tabla de estados de vuelos.

Acá el primer campo indica el aeropuerto de destino.

Sobre qué representa cada columna:
Airport: Aeropuerto de destino
Flight: Número de vuelo
Date: Fecha del vuelo
Origin: Aeropuerto de origen
Status: Estado del vuelo
Timestamp: Marca cuando se hizo la última actualización a la tabla

Partición: airport,date.
Primary key: airport, date, flight.

- **`flights_in_progress`** table:

flight	airport	destination	status	date	speed	latitude	longitude	height	timestamp

Tabla de simulación de vuelos, donde se va actualizando dinámicamente sus campos.

Esta instancia de tabla existe para cada uno de los nodos donde guardará la información sobre cada vuelo, de aquí obtiene sus coordenadas a cada instante, además de actualizaciones sobre el estado de vuelo.

Sobre qué representa cada columna:
Flight: Número de vuelo
Airport: Aeropuerto de origen
Destination: Aeropuerto de destino
Status: Estado del vuelo
Date: Fecha del vuelo
Speed: Velocidad del vuelo
Latitude, longitude, height: latitud, longitud y altura del vuelo
Timestamp: Marca cuando se hizo la última actualización a la tabla

Partición: airport,date.
Primary key: airport, date, flight.

Tablas de soporte

Estas tablas no se crean utilizando CQL, además de no tener copias por nodos, simplemente se utilizan para dar soporte a, principalmente, la interfaz gráfica.

- **`airports`** table:

airport	country	longitude	latitude

Tabla que guarda información sobre aeropuertos

Esta tabla simplemente guarda el código de aeropuerto, país del que forma parte, longitud y latitud para poder ser utilizados en la interfaz gráfica además de tener una referencia de la ubicación de los aviones.

- **`users`** table:

username	password	role

Tabla que guarda información sobre los usuarios

Esta tabla guarda todos los usuarios existentes para poder loguearse además de su rol, el rol puede ser:

Admin: Tiene acceso a todas las operaciones CRUD.
User: Solo puede ver los vuelos pero no puede modificarlos.

- **`logs_<n1>_<n2>`** table:

Estas tablas son simplemente un archivo en blanco, temporal, donde se van guardando las consultas para el hinted-handoff. Se crean solamente cuando es necesario.
<n1> representa al nodo que manda la consulta. mientras que **<n2>** es el nodo a quien se les manda. Más adelante se explicará mejor su función.

2. Operaciones CRUD

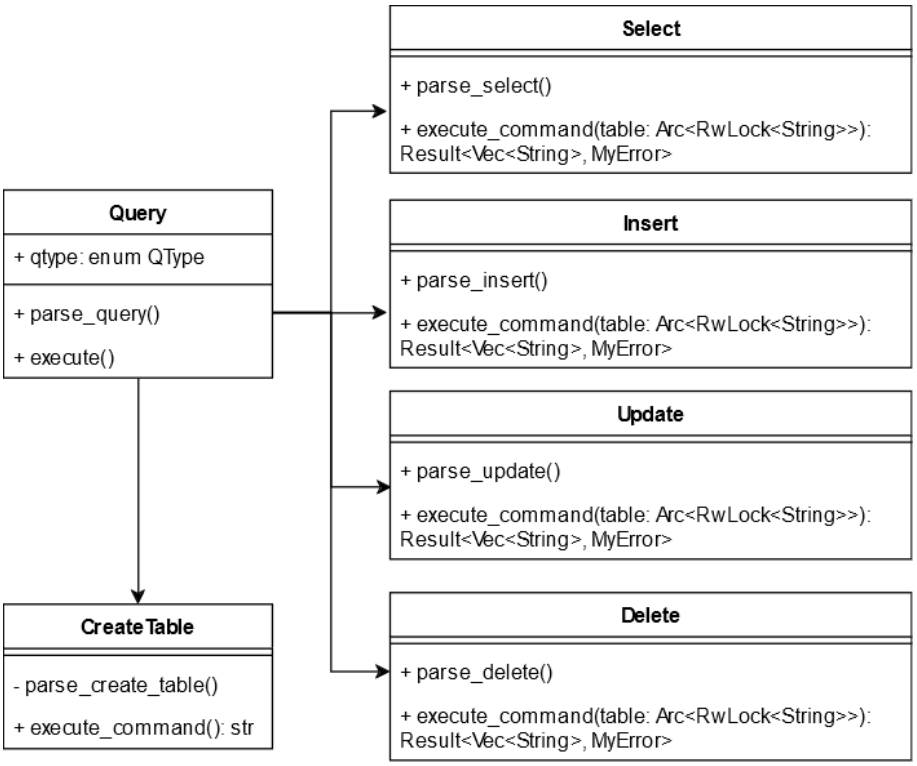
- Operaciones de creación, lectura, actualización y eliminación utilizando CQL (Cassandra Query Language)

Nuestro servidor admite 5 operaciones de CQL: Select, Insert, Update, Delete y Create Table.
Cuando el Cliente le manda al Nodo una consulta CQL, este crea una entidad del tipo Query que parsea la consulta y también devuelve su respectivo partition key.
Con este partition key, el Nodo sabe a quién mandarle la Query, para que el correspondiente Nodo luego pueda ejecutarla y devolver el resultado. La ejecución del comando debe primero

recibir un Mutex de la ruta al archivo, esto se hace para que diferentes threads no puedan acceder y editar al archivo al mismo tiempo ya que esto podría producir errores no deseados como un error de Read o que simplemente el archivo no devuelva lo esperado.

Es importante notar que Create Table es ligeramente diferente a las otras consultas. Este es ejecutado solamente por el Nodo que recibe la consulta del cliente, y además este no necesita recibir ningún Mutex ya que es el que crea las tablas en sí.

Otra observación, aunque técnicamente solamente el Select debería devolver registros de la tabla, todas las consultas devuelven un vector de Strings que aseguran que el comando se ejecutó con éxito, de no ser así se levanta alguno de nuestros errores personalizados.

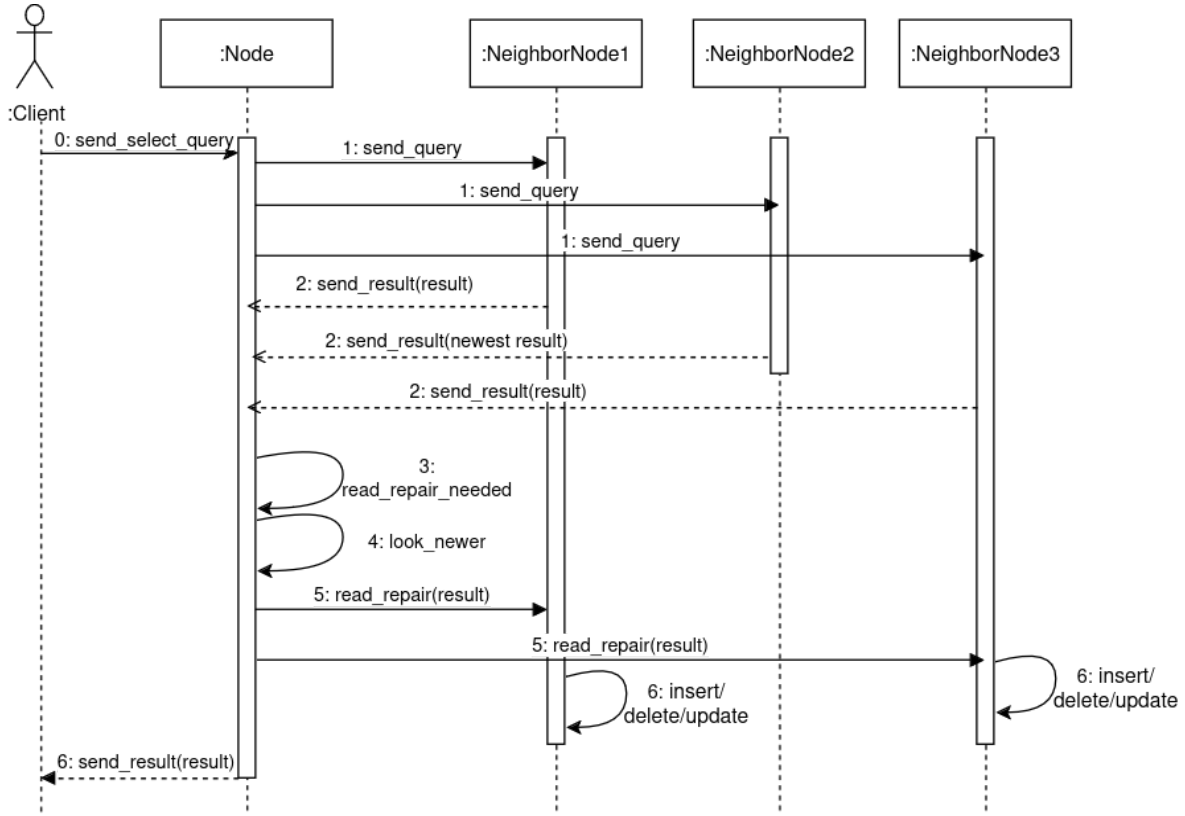


○ Implementación de Read-Repair

Cuando el cliente realiza una consulta Select, el Nodo principal le delega la consulta a los Nodos requeridos y espera una respuesta. Cuando ya termina de recibir respuestas (las cuales pueden ser todas, algunas o ninguna), el Nodo decide si necesita mandar un Read-Repair. Esto lo hace en la función **`process_results()`**. Si hay menos de dos respuestas, Read-Repair ya es descartado, y si las dos o más respuestas son diferentes (sin contar los Timestamps) se busca cual es la respuesta acertada con **`look_newer().`** En esta función, se busca la respuesta más actualizada, y luego se le manda el resultado dentro de un **`ReadRepairMessage`** a los Nodos desactualizados.

Cuando le llega un `ReadRepairMessage` a un Nodo, este va a **`handle_message_read_repair()`** donde decide si es necesario hacer Updates, Deletes o Inserts.

Al mismo tiempo que están ocurriendo los Read-Repair dentro de los otros Nodos, el Nodo principal le manda la respuesta del Select al Cliente.



3. Cliente - Servidor

En este proyecto existen 2 tipos de cliente, uno que hace se comunica con el servidor mediante la terminal y otro que utiliza la interfaz gráfica, este último va a ser explicado más a detalle posteriormente, ahora nos enfocaremos un poco en el primero y luego en una idea general de cómo funcionan los clientes y el servidor.

El cliente “por terminal” escribe sus consultas dentro de un thread mientras lee del proceso principal, las funciones permitidas son:

- Create Table
- Select
- Update
- Delete
- Insert

En caso del Select, recibirá la lista de datos pedida como es de esperar, para las otras consultar recibirá simplemente una confirmación sobre la modificación/creación de la tabla. En casos donde la consulta falle o no se pueda procesar, el cliente recibirá un error para notificarlo.

El cliente (tanto el que usa la terminal como el de la interfaz gráfica) no sabe qué nodos tienen la información, cuales estan caidos o levantados, simplemente hace una consulta y esta es respondida

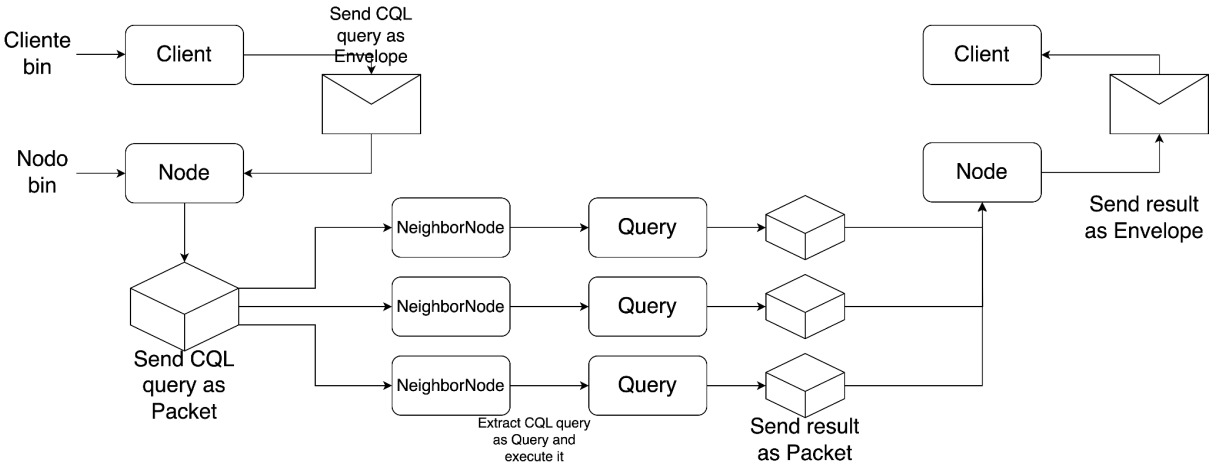


Diagrama de comunicación cliente - servidor

Como vemos en la imagen, cuando el cliente hace una consulta, esta se guarda en un Envelope, esto se detalla luego en la parte 5 pero a simples rasgos, la consulta se pone dentro de una estructura que sigue el protocolo nativo de cassandra, esta luego es leída por el nodo con el que se comunica, este analiza la consulta, obteniendo los datos necesarios a hashear (aeropuerto y fecha) para saber que otros nodos vecinos deben realizar la consulta (también puede ser el uno de esos nodos), esto se hace justamente enviando la consulta de como un Paquete el cual es una forma reducida del protocolo de cassandra. Los nodos, de estar conectados y conocidos por el nodo comunicador mediante gossip, obtendrán el Packet, extraerán la consulta a hacer y la ejecutarán en sus tablas correspondientes, ahora devolviendo el resultado de la consulta al nodo comunicador. Este analiza las respuestas obtenidas, realizando cuantas respuestas le llegó, si hace falta hacer uso del hinted handoff, en caso de un Select, revisará cuál es la respuesta más nueva para realizar un read repair a los demás nodos de ser necesario. También tiene en cuenta el consistency level, por ejemplo en el simulador de vuelos donde hay Weak consistency, el nodo comunicador devuelve la primera respuesta que le llega y no le interesa el resto, en Strong consistency, espera a que todos terminen.
Una vez obtenida la respuesta, esta se guarda en un Envelope como al principio, mandando al cliente para que pueda saber el resultado de su consulta.

4. Replicación y Consistencia

○ Decisiones de particionamiento

Se tomó la decisión de particionar las tablas, tanto de Estados de Vuelos, como del Simulador de Vuelos por los valores del aeropuerto principal que representa la tabla y el valor de la fecha en la cual se desea ver los vuelos.

Ya que se consideró la necesidad de hacer consultas sobre el estado de los vuelos, tanto por el aeropuerto de origen como el de destino, se optó por replicar la información en dos tablas. Una tabla que representa a los vuelos con su fecha y aeropuerto de origen (llamada *origin table*), y otra donde se identifica a los vuelos por la fecha y su aeropuerto de destino (*destination table*).

En resumen, se cuenta con las siguientes tablas:

- origin table:

airport	flight	date	destination	status	timestamp

Tabla de estados de vuelos.
El primer campo indica el aeropuerto de origen.
Partición: airport,date.

- destination table:

airport	flight	date	origin	status	timestamp

Tabla de estados de vuelos.
Acá el primer campo indica el aeropuerto de destino.
Partición: airport,date.

- flights_in_progress table:

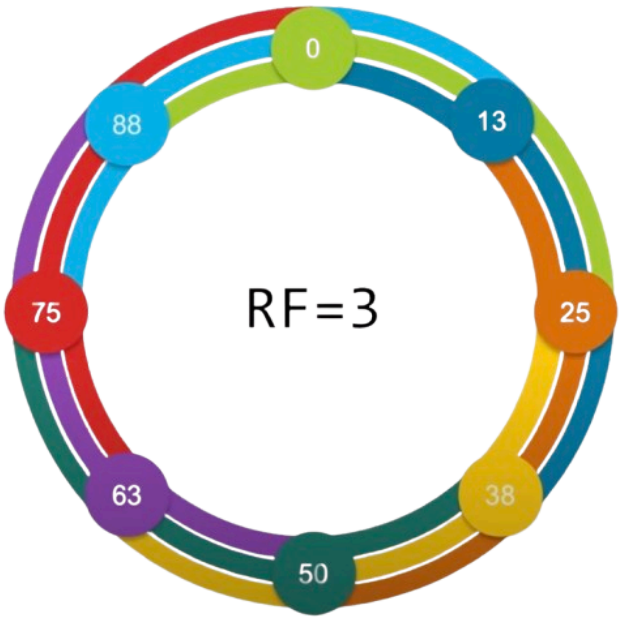
flight	airport	destination	status	date	speed	latitude	longitude	height	timestamp

Tabla de simulación de vuelos, donde se va actualizando dinámicamente sus campos.
El primer campo indica el aeropuerto de origen.
Partición: airport,date.

○ Configuración de estrategias de replicación (Replication Factor)

Se realizó la replicación de datos en más de un nodo según el valor de Replication Factor (RF), el cual indica la cantidad de nodos en la que se guardarán los valores de un vuelo. La elección de este valor fue debido a que la posibilidad de que 3 servers fallen al mismo tiempo

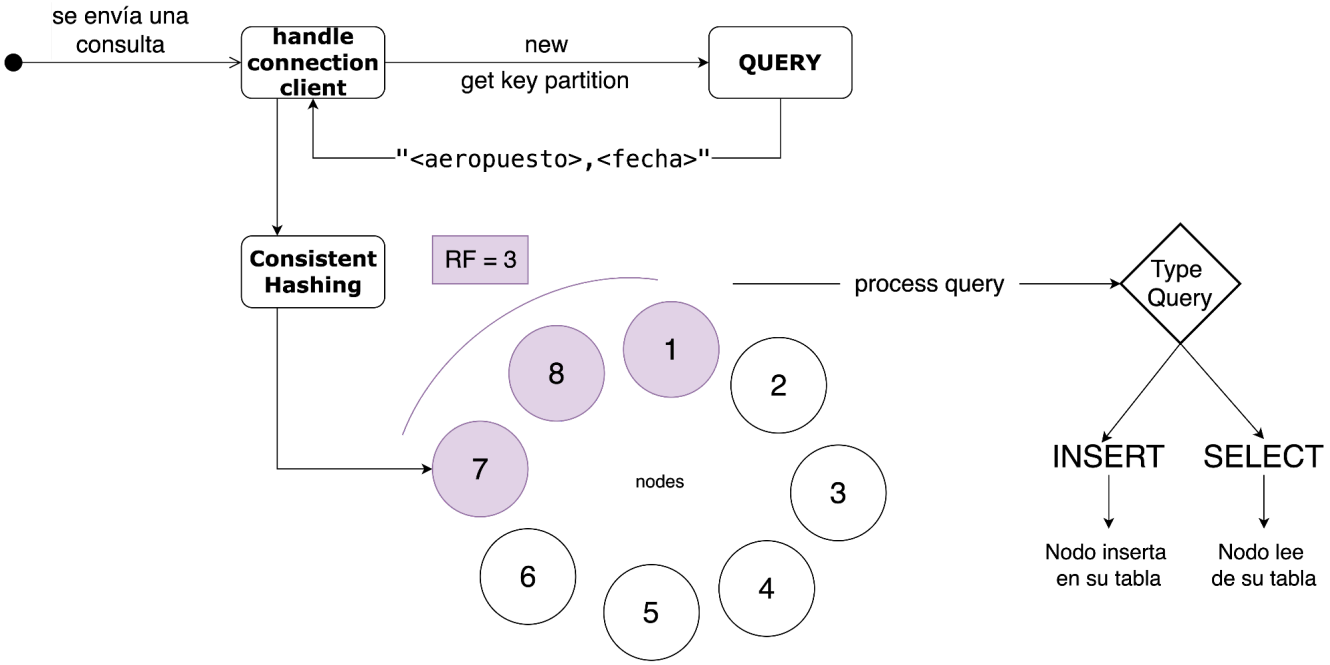
es baja.



En el diagrama anterior, se puede observar que contando con 8 nodos (como fue nuestro caso), dado un valor entre 0 y 100 que se desea guardar, éste se replicará en tres nodos representados por el color en el que está pintada la curva que los contiene.

- Consistent Hashing

El flujo en que se realiza la elección de los nodos en los que se guardará la información, es el siguiente:



En primer lugar, se debe asignar un valor de hash a cada nodo, este se calcula a partir del valor de la address del nodo y la normalizamos por la cantidad de nodos. Se los sitúa a los nodos en el ring dependiendo del valor que recibe. (Aclaración: la posición en la que se ubica un nodo no tiene porque coincidir con el orden de sus números de address (IP:puerto)).
Luego, cuando llega una consulta, se obtienen los valores de los campos que representan las particiones y se les calcula su valor de hash a partir de este valor pasado a string (**“<airport>,<date>”**). Con este valor, nos posicionamos en el ring y tomamos el nodo más cercano en el sentido de las agujas del reloj. Éste será el primer nodo que contendrá la información de la consulta, y los dos siguientes los otros nodos en los que se replica la información.

○ Soporte a distintos niveles de consistencia (Consistency Level)

Nos encontramos con la posibilidad de que las tablas puedan tener uno de estos dos niveles de consistencia (CL): ONE or QUORUM.

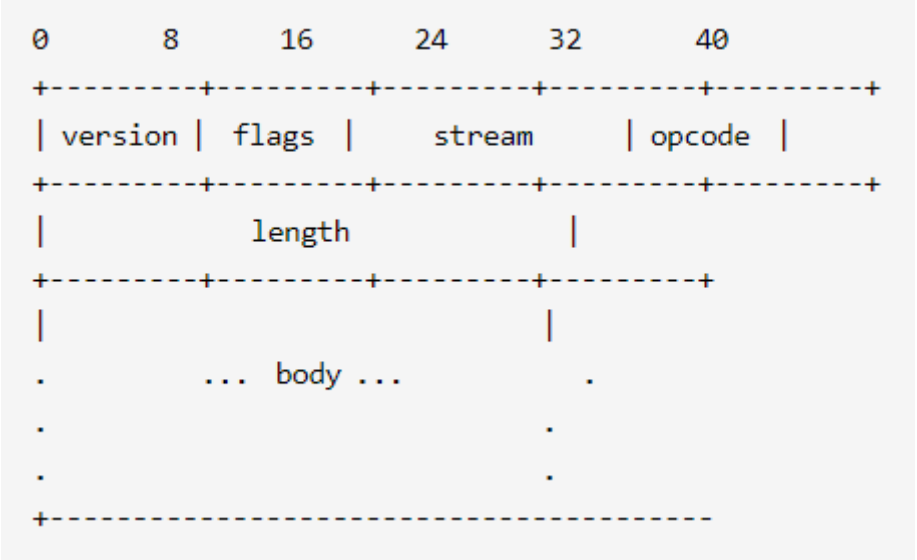
- En el caso de **CL = ONE**, al iterar entre los nodos que tienen la información, se devuelve el resultado que devuelve el primer nodo por el que se itera sin procesar los otros. De esta forma se tiene un proceso más rápido, por lo que es el adecuado para la tabla que se usa en el Simulador de Vuelos con weak consistency.

- En el caso de **CL = QUORUM**, se obtienen los resultados de todos los nodos en los que se replica la información y se devuelve el más actualizado, es decir el que posee el mayor timestamp (que es el último campo de todas las tablas). Este es usado en las consultas sobre las Tablas de Estado de los vuelos (origin y destination).

5. Protocolo Nativo de Cassandra

○ Tipos de mensajes

La comunicación entre el cliente y el servidor se hace ‘envolviendo’ la respuesta o la consulta en una estructura llamada Envelope el cual sigue el protocolo Nativo de Cassandra



Los tipos de mensaje que se implementaron para el proyecto son el de Consulta, Resultado y Error. Estos se ven reflejados tanto en la versión como en el opcode. Viendolo mas en detalle:

Consulta

Lo utiliza el cliente para realizar una consulta CQL, contiene tanto la consulta como el consistency level.

Error

Cuando algo salió mal por parte del server, devuelve un error al cliente, contiene tanto el String con el error como un código identificador

Resultado

Lo utiliza el servidor para dar aviso del resultado, en una operación de tipo Select, devuelve la tabla pedida, en otro tipo de operaciones como crear una tabla o el resto de operaciones CRUD, devuelve una confirmación de que la modificación fue hecha.

6. Funcionamiento de Clúster

○ Implementación del protocolo Gossip para comunicación entre nodos

Los nodos utilizan los puertos 9000 para el protocolo Gossip, esté en simples términos funciona de la siguiente manera:

Cuando se levanta un nodo:

El nodo recién levantado comunica su existencia a otro nodo (elegido aleatoriamente), ahora estos 2 comunican la existencia del primero a otros nodos y va eligiendo hasta que todos sepan de su existencia, este mensaje no se repite infinitamente sino que tiene dentro una variable donde, después de repetirse 5 veces, deja de mandar el mensaje.

Cuando cae un nodo:
Como el nodo puede caer repentinamente, este no puede dar aviso de que se va a caer, alguno de los nodos que esta conectado con el nodo caído, se dara cuenta porque intentara realizar una coneccion y no podrá establecerla, de suceder esto, el nodo que se da cuenta del fallo, manda por gossip un aviso al resto de nodos para que actualicen su información y sepan que el nodo está caído (Esto sirve además para activar hinted-handoff cuando llega una consulta que modifica tablas al nodo que está caído).

En caso de que el nodo caído vuelva a conectarse, comunicará su existencia como si recién se hubiese levantado, en caso de que tenga consultas que hacer por hinted-handoff, cuando el nodo que posee las consultas a realizar sepa de que el nodo volvió a conectarse, empezará a mandar las consultas para que las ejecute (cabe aclarar que esto último lo manda por los puertos de comunicación entre servers (7000) y no por el puerto de gossip que existe solo para comunicar existencia de nodos, no para mandar información sobre consultas)

- ## Tolerancia a fallo de uno o más nodos del cluster

Ante la caída de uno de los nodos. No se modifica el ring de nodos y sus respectivas valores del hash. Esto da la posibilidad de que cuando se restablezca el nodo, posea el mismo hash que tenía anteriormente.

- ## Implementación de Hinted Handoff para recuperación de datos ante la desconexión de un nodo

Por gossip, un nodo sabe si otro está conectado o si está caído. Cuando el cliente manda una consulta, específicamente una consulta que modifica campos (update, delete, insert) y uno de los nodos que debe realizar esa consulta está caído, el nodo comunicado con el cliente, guarda en un registro (logs) la consulta a hacer. Posteriormente, cuando el nodo caído vuelva a levantarse y comunique su existencia a otros mediante gossip, el nodo comunicador leerá el registro guardado y le mandara todas las consultas a hacer, de esta manera el nodo recién levantado se “pone al corriente” actualizando hasta la ultima consulta que necesita, como datos extra, estos registros persisten en archivos de texto plano que una vez actualizado el nodo, son eliminados.

- ## Punto Bonus (opcional): demostrar la compatibilidad utilizando un cliente comercial de Cassandra. *Próximamente…*

7. Seguridad

○ Autenticación y autorización

Tenemos un sistema bastante simple de autorización y autenticación que se utiliza solamente en la interfaz gráfica.
Cuando un cliente abre la aplicación, se va a encontrar con un mapa con el que no puede interactuar, para poder hacerlo primero debe ingresar un usuario y su contraseña, los cuales ya fueron agregados anteriormente al archivo `users.csv`. Una vez que se valide su identidad, la interfaz gráfica se amolda dependiendo del rol que tiene el cliente: Administrador o Usuario.
Si se tiene rol de Administrador, el cliente va a poder ver, agregar, sacar y actualizar los vuelos.
Si se tiene rol de Usuario, el cliente solo va a poder ver los vuelos.
Es importante notar, que para poder ver cualquier vuelo, primero se debe agregar una fecha antes de cliquear algún aeropuerto.

○ Encriptación de datos en tránsito (in-transit)

Para encriptar los datos de los mensajes, simplemente usamos una librería: AES. Implementamos una función que encripta y otra que desencripta, ambas usando la misma clave por simplicidad. Estas funciones trabajan con vectores de u8 lo que facilita el manejo de bytes individuales. Luego, en la creación tanto de los Envelope como de los Packets en sus respectivas funciones, **`to_bytes()`**, se llama a la función encriptar justo después de obtener el Payload del mensaje, en bytes, de esta manera el header queda sin encriptar y el contenido del mensaje es que se encripta.

○ Encriptación de datos en reposo (at-rest)

Siguiendo con la encriptación de datos in-transit, hicimos algo parecido con nuestras bases de datos. Volvimos a utilizar la librería AES.
Elegimos una única llave de largo 128 bits que usarían todos los nodos de forma simétrica.

Tuvimos que tener más cuidado con los archivos, ya que dependiendo como encriptabamos, esto nos dificultaba luego la lectura de archivo, la cual es esencial para ejecutar los comandos CQL. Por lo tanto, decidimos que además de encriptar los registros, también los terminamos encodeado a los siguientes valores: "**`0123456789ABCDEF`**". Esto nos ayudó a dividir los registros con saltos de líneas sin tener que preocuparnos a que la encriptación les agregue un salto de línea de por medio.

Por lo tanto, el formato de nuestros archivos encriptados es el siguiente:

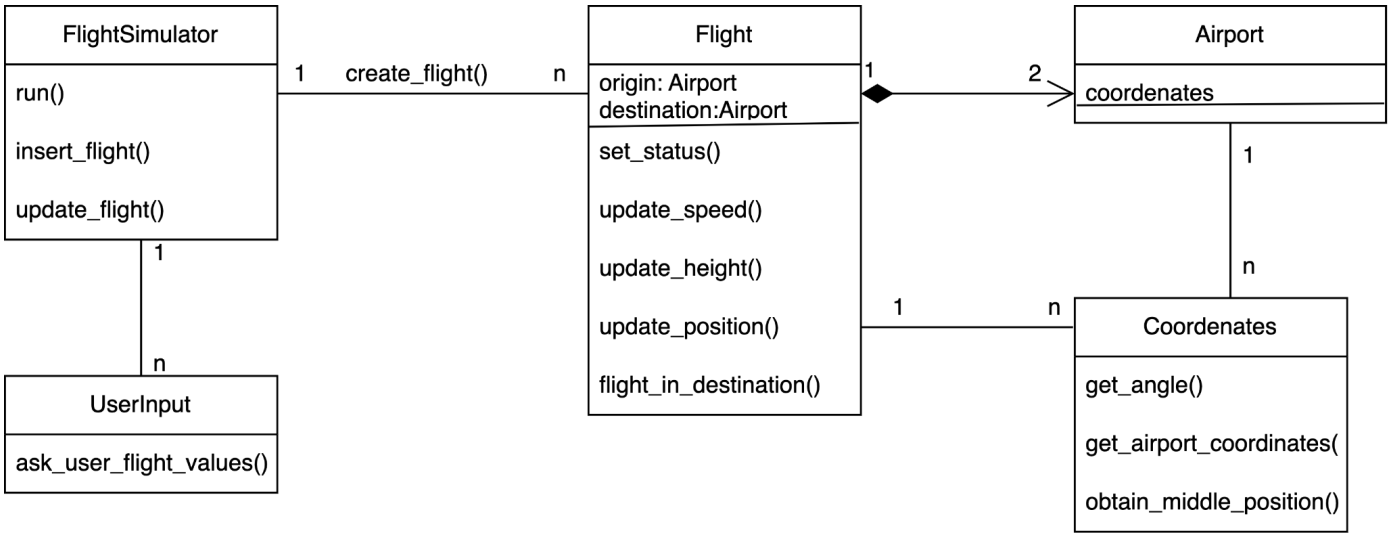
- Cada línea representa un único registro, incluyendo las comas que separa cada valor. Todo encriptado.

- La primera línea son los nombres de las columnas, separadas por comas. Todo encriptado.
- Decidimos no encriptar el archivo de metadata.

Una última observación: si se quiere sacar la encriptación at-rest, solamente basta con setear la constante **`ENCRIPT_DATA`** a falso en el archivo **`config.rs`**.

8. Simulador de Vuelos

A continuación se pueden ver los distintos módulos que se necesitaron para el simulador:



Se implementó la estructura de **`FlightSimulator`** la cual es llamada desde el binario `src/bin/simulator.rs` llamando a la función run().
Se pueden simular tantos vuelos como se quieran, cada uno se simulará en un thread aparte.
En cada uno de estos hilos, se pide al usuario que ingrese por consola la información del vuelo que se desea simular. Con esta información se crea cada uno con la estructura Flight. Para esto, se necesita obtener las coordenadas de los aeropuertos de origen y destino que se encuentran en la tabla **`src/tables/airports.csv`**. También se calcula el ángulo en el que se desplazará el avión dependiendo del eje x. Esto resultó importante para saber si se debe incrementar o decrementar cada una de las coordenadas de latitud y longitud para que el avión avance hasta el destino.
Cada uno de los vuelos es agregado a la tabla **`flights_in_progress`**, y también en las tablas de estado **`origin`** y **`destination`** en caso de no encontrarse ya.
Por último, se itera hasta que el avión haya llegado a destino, actualizando cada uno de los parámetros que describen cada posición del vuelo en su recorrido.
Los parámetros a actualizar fueron:

- **`status`**:
    - Se inicia con estado **`Scheduled`**.
    - Cuando está por cambiar de la posición de origen cambia a **`Boarding`**.

    - En la siguiente iteración cambia a **`Airborne`**. Saliendo ya el avión de la posición de origen. Y se mantiene de esta forma mientras su posición no sea la de destino.
    - Por último, al llegar a destino se cambia a **`Landed`**.
- **`speed`**:
    - Se crea el vuelo con velocidad 0.
    - Se aumenta la velocidad del vuelo hasta llegar a la cercanía del punto medio (rango medio).
      Aclaración de términos utilizados:
        - *Steps: Cantidad de pasos que realiza el avión hasta llegar al punto medio del recorrido. También representa la cantidad de pasos entre el punto medio y el destino.*
        - *Un step: Distancia en km que recorre el avión por cada iteración.*
        - *Rango medio: punto medio ± 2 steps*.
    - En el rango medio se tiene que llegar a la velocidad promedio recibida (También llamada velocidad crucero).
    - Pasado el rango medio debemos disminuir la velocidad de forma tal que al llegar a la posición de destino la velocidad sea 0 km/h.
- **`latitude y longitude`**:
    - Fue necesario calcular la distancia que debe recorrer en cada iteración para llegar en los STEPS indicados de origen a destino, tanto en el eje x como en el y. Nosotros tomamos la decisión de que el vuelo se actualice 20 veces para llegar desde el origen hasta el destino, siendo este un valor que se puede modificar con facilidad. A partir de esto se calcula lo que se debe avanzar en cada paso en cada coordenada.
    - Se inicia con la latitud y longitud del aeropuerto de origen.
    - Se actualiza cada coordenada dependiendo del ángulo en el que se mueve el avión.
    - Se termina en las coordenadas del aeropuerto de destino,
- **`height`**:
    - Al igual que la velocidad, empieza en 0 m y aumenta hasta el rango medio, donde mantiene una altura crucero. Luego de esto disminuye hasta llegar a 0 m en el aeropuerto de destino.

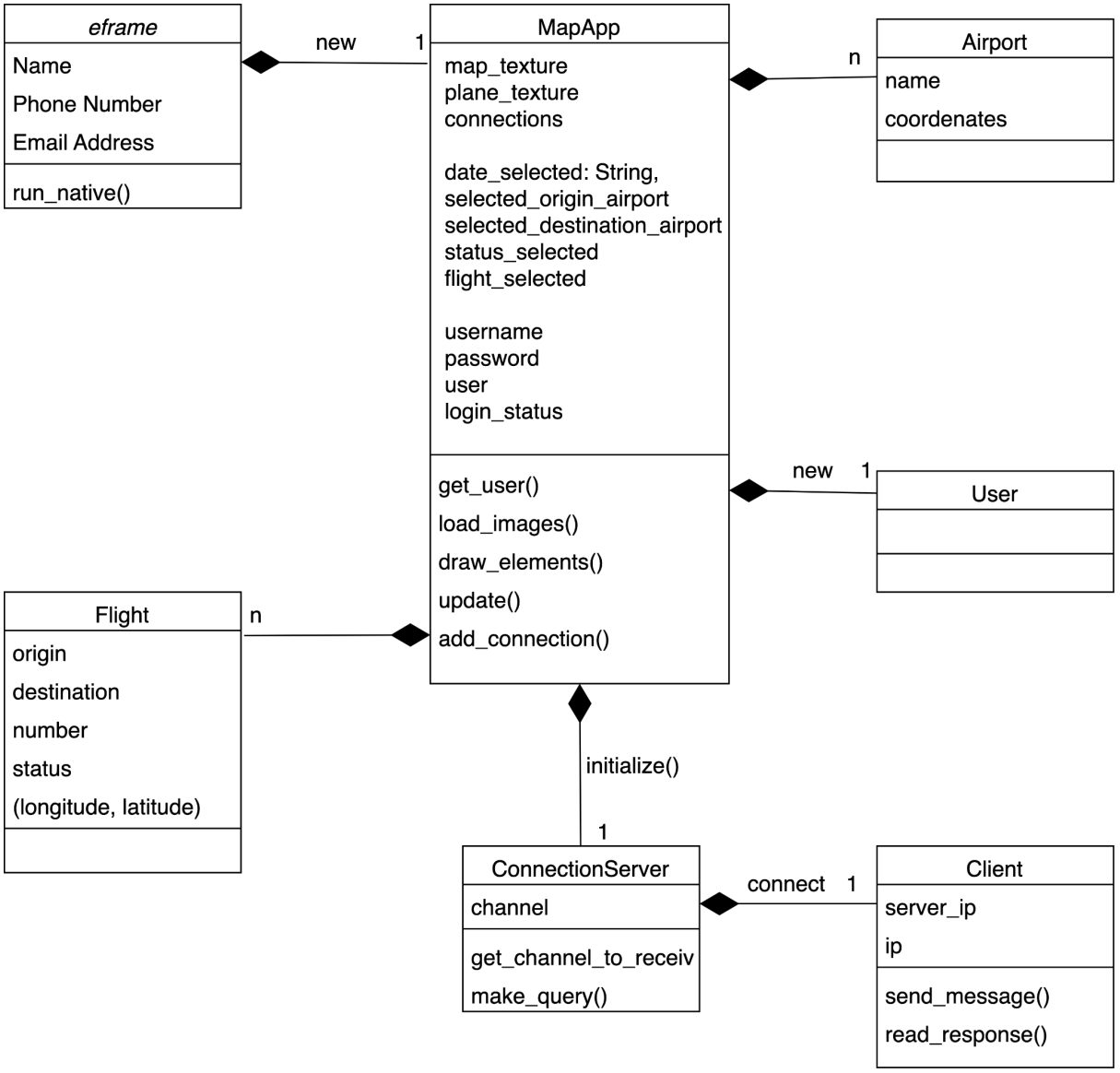
Para la ejecución de las queries se está utiliza **`Thread Pools`** para ejecutar las queries de UPDATE de los vuelos.
Se están encolando las queries en el orden que se quieren hacer en una “cola”. Decidimos no utilizar precisamente una estructura de Queues sino que utilizamos un archivo el cual se lockea para su escritura (enqueue) y para obtener la primera query del archivo (dequeue).
Para esto tenemos la estructura **`FlightUpdater`**. A partir del cual lanzamos un thread, el cual se encargará de escuchar por un channel las queries que se desean realizar y las encolará. Y otro thread que estará lanzando los Thread Pools donde cada uno desencolará una query del archivo y la enviará al server.
Desde el modulo del Simulador tendremos uno de los extremos del channel por el que envíamos las queries de update.

9. Interfaz gráfica

○ Módulos

Se utilizó **`egui`** versión "0.29" para la realización de la GUI.

Punto de inicio de la ejecución: `src/bin/gui.rs`.

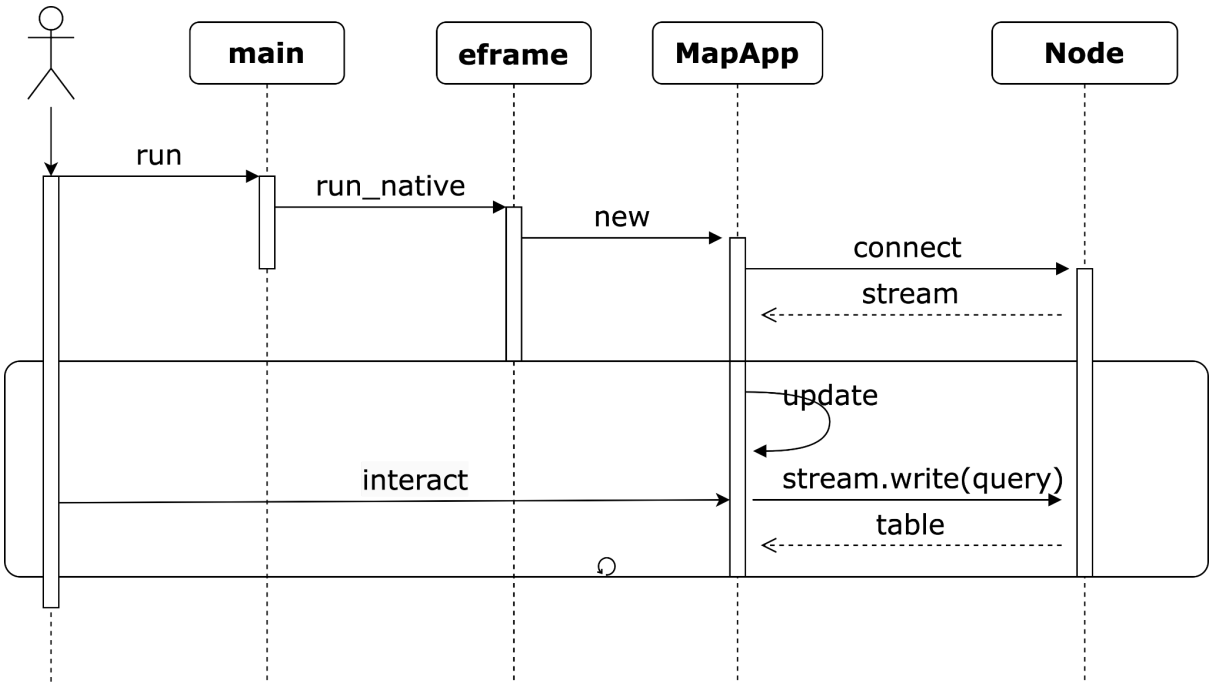


Desde el egui framework crate **`eframe`** se pudo realizar la interfaz del usuario. Al llamar a la función **`run_nativa()`** está nos inicia nuestra desktop app, y debemos pasarle nuestra aplicación **`MapApp`**.
Desde nuestra app nos conectamos como clientes al servidor a través de **`ConnectionServer`**, y podremos hacer las queries necesarias para agregar

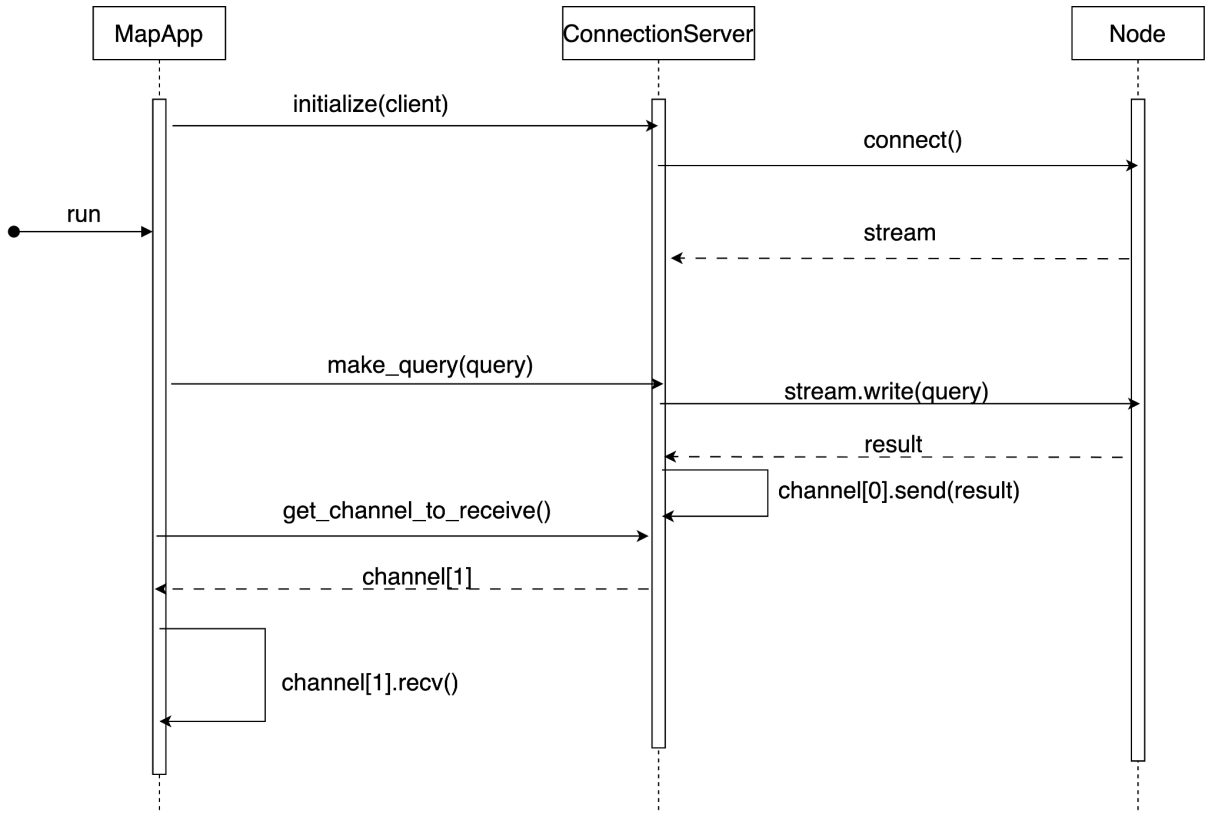
comunicaciones entre aeropuertos a través de un **`Flight`**. Estas conexiones serán guardadas en el campo **`MapApp::conecctions`**.
Para poder visualizar las conexiones entre aeropuertos se necesita loguearse el usuario.
La función **`update()`** es donde se cargan las imágenes, se posicionan los aeropuertos y vuelos, y se interactúa con el usuario.

- ## Ver vuelos y editar el Estado de los vuelos

En primera instancia, se realizó todo el flujo para obtener los vuelos utilizando un único thread que realizaba tanto la renderización de las imágenes como las consultas al servidor.
Se podía observar las siguientes interacciones entre los distintos módulos:



Esto hacía que el flujo sea pesado en el único thread que contábamos, por lo que lo modificamos por dos threads, uno encargado de la renderización y actualización las imágenes que queremos mostrar al usuario, y otro encargado de las conexión y consultas al servidor.
Ambos threads se comunicarán por un channel. Siendo el thread de conexión con el server el que envía el resultado de la consulta por el channel, donde el thread principal lo lee.
Llamamos al módulo encargado de hacer las queries **`ConnectionServer`** ubicado en la carpeta de la UI (**`/gui`**).
Se observa ahora el siguiente flujo al hacer las consultas:



○ Integración del simulador

Contando con el módulo **`ConnectionServer`** pudimos agregar un paso al calcular la posición en la que posicionamos el vuelo. Esto fue, hacer una consulta SELECT buscando el vuelo que queremos mostrar en la tabla del Simulador de vuelos. En el caso de que se encuentre, se dibuja en la posición que indica esta tabla, caso contrario en la mitad del recorrido.
Además, agregamos un refresh al final de la función update() de MapApp. Con egui lo hicimos con: **`ctx.request_repaint();`**.
Por último, sólo necesitamos rotar las imágenes del avión para poder apuntarlo en la dirección de origen a destino.

10. Links útiles

Cassandra
- Protocolo Nativo de Cassandra
- CQL
- Consistency Hashing
- Read Repair
- Hinted Handoff

Concurrencia
- threads
- locks

- Arc
- Thread Pool
- Crate threadpool
- Channels

Sockets
- TcpStream
- TcpListener

GUI
- egui
- eframe
- walkers

Seguridad
- aes

Final 2C 2024: Containerization

1. Introducción

A partir del servidor distribuido desarrollado durante el cuatrimestre “Aerolíneas Rústicas”, lo implementamos en una serie de contenedores de Docker.
Utilizamos un contenedor para cada nodo del cluster de base de datos de tipo Cassandra.
Además, se incorporó la posibilidad de hacer más flexible la distribución de nodos del cluster.
Por último, se agregó un logger de intercambio de mensajes entre nodos.

2. Docker

○ Docker-compose

Este archivo configura múltiples nodos (contenedores) que forman parte de la red de nodos de la base de datos distribuida.
Cada nodo representa un servicio que utiliza la imagen **`rust:latest`** y se ejecuta en contenedores con direcciones IP definidas dentro de la red, permitiendo la comunicación entre ellos.

Mediante el uso de un pequeño script the bash y un template, generamos un archivo .yml que sirve para correr varios nodos, además de que utilizan una network para comunicar los nodos entre sí.
Para comunicarse con el cliente, cada nodo mapea el puerto 8000 del container a un puerto 80XX de localhost de la máquina donde se está corriendo.

Utilizamos **`volumes`** para visualizarlo mejor. En este campo se mapea lo que genera cada container a nuestra máquina para poder visualizar todos los archivos de los containers (tablas, logs, etc.).

- ### Comandos

Para crear n nodos usamos:

```
./start.sh n
```

Para añadir un nodo mas a la red:

```
docker run -d --name node7 --network network_net --ip 10.0.0.7 -p 8007:8000 -v $(pwd):/app -w /app rust:latest bash -c 'apt-get update && apt-get install -y iputils-ping && cargo run --bin server 10.0.0.7 10.0.0.5'
```

Tener en cuenta que es una demostración, para levantar cualquier nodo hay que cambiar el 7 por otro número, la última ip presente es la ip a la que intentará conectarse.

Para utilizar el cliente/gui/simulador:

```
cargo run --bin client 127.0.0.1 800n

cargo run --bin gui 127.0.0.1 800n

cargo run --bin simulator 127.0.0.1 800n
```

Para eliminar un nodo usamos las herramientas de docker desktop, tanto el stop como el delete.

La diferencia entre la caída de un nodo y la desvinculación tiene que ver con la cantidad de tiempo desconectado, no con las funciones de docker.

3. Reconfiguración dinámica del cluster

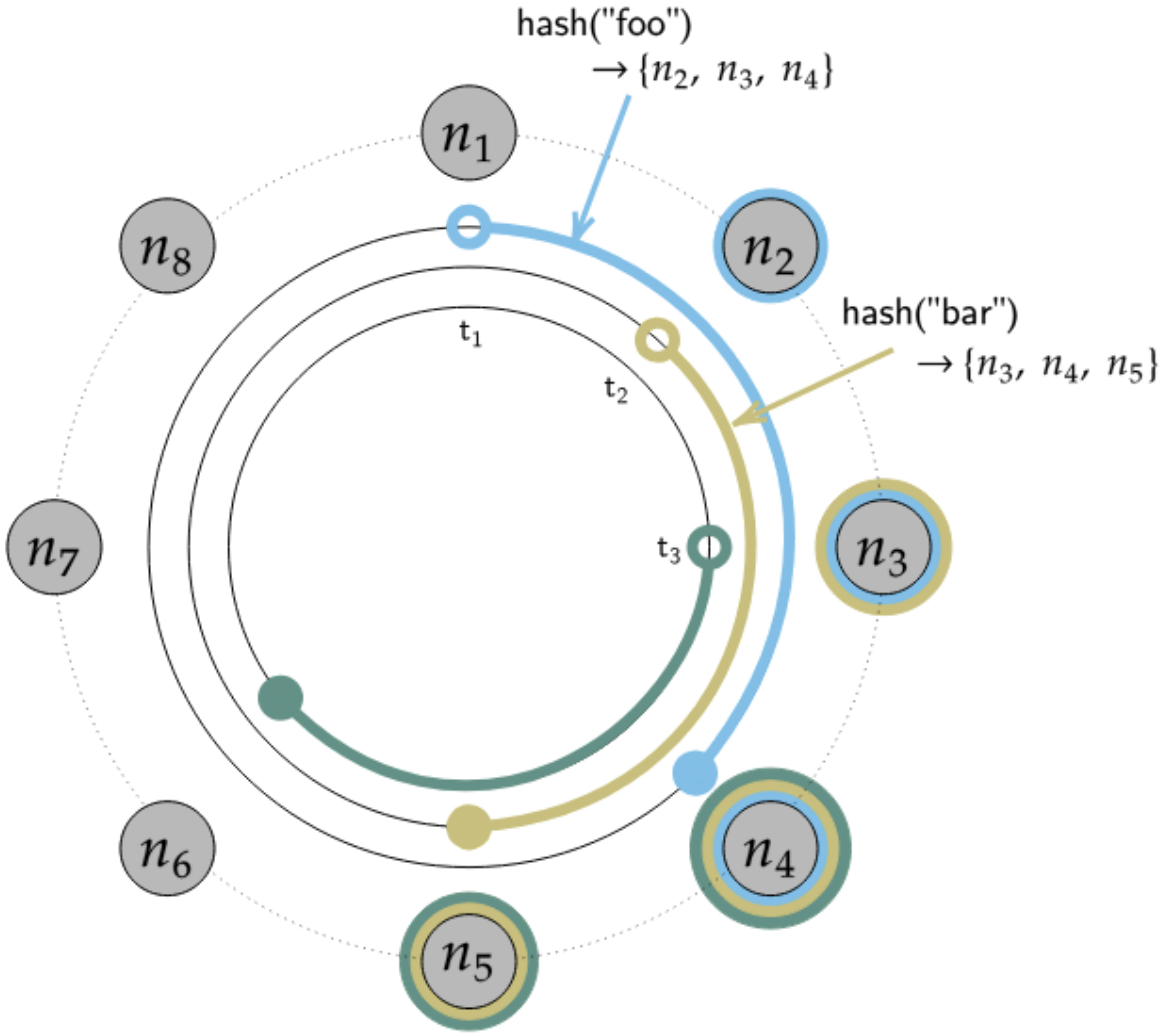
Realizamos las modificaciones necesarias para contar con N nodos no fijos. Por lo que nuestro proyecto funcionará con la cantidad de nodos deseada, y se podrá agregar un nuevo nodo a la red luego de que ésta esté en funcionamiento. Además, sigue soportando la caída de un nodo pero con una mejora: ahora los datos que debería recibir este nodo se distribuirán entre los demás. De esta forma, no perdemos la cantidad de replicaciones de los datos.

Antes de implementar el agregado y desvinculación de un nodo, realizamos las siguientes mejoras en nuestra implementación:

a. Ampliación del Ring:

Nuestro ring era de 8 nodos fijos, lo cual nos traía problemas ya que al realizar el módulo del hash de un nodo este podía caer en la misma posición que otro. Para esto nosotros incrementamos en 1 su posición pero no nos permitía obtener los mismos nodos dada una

key en aisladas ocasiones. Es por esto que decidimos ampliar significativamente nuestro anillo de nodos, previniendo de esta manera que dos nodos caigan en la misma posición.
Ahora no contamos con un anillo en el que en todas las posiciones están ocupadas por un nodo físico, sino que cada nodo tiene un rango del anillo que le pertenece como se observa en la siguiente imagen.



Acá podemos observar que dada la key “food”, que cae en una posición del anillo en la que no se cuenta con un nodo, se debe buscar la próxima posición más cercana en el sentido de las agujas del reloj, resultando el nodo n2. Luego los otros dos nodos que contengan la información serán las próximas dos posiciones del anillo ocupadas por uno en el sentido dicho anteriormente. Es decir, las réplicas de esta información estarán en los nodos n3 y n4.

b. Múltiples tokens por nodo físico (vnodes):

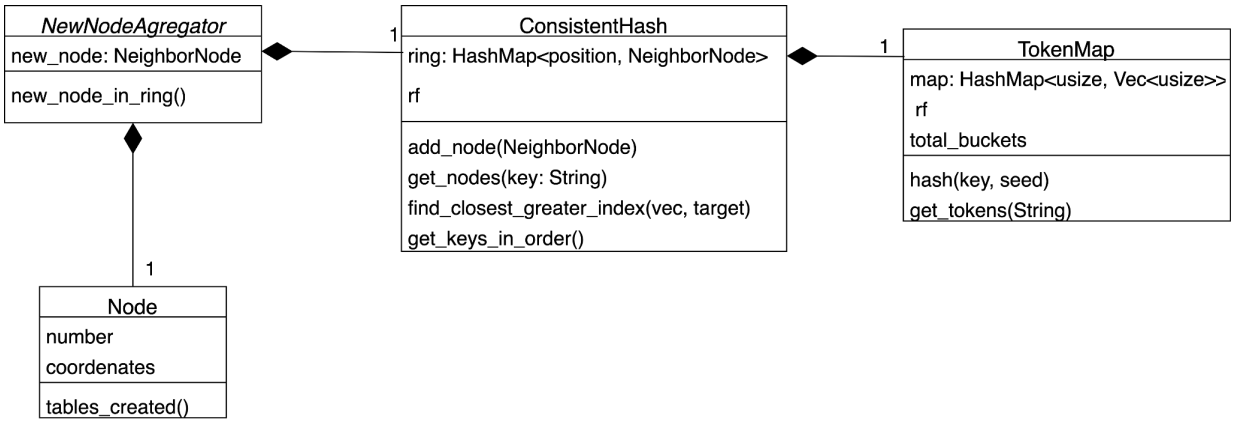
Incorporamos la utilización de RF nodos virtuales por cada nodo físico. Esto quiere decir que dado un nodo físico lo posicionamos 3 veces en el anillo de nodos, cada uno de estos será un vnode.
Agregamos al **`Consistent Hashing`** un **`Token Map`** que mapea todas las posiciones que ocupan los vnodes de un nodo físico en específico.
Tener nodos virtuales nos ayudan a tener una carga de consultas equilibrada entre los nodos físicos.

En este caso, cuando queremos obtener los 3 nodos en los que se replica la información sólo debemos asegurarnos de que no se repitan.

○ Incorporación de un nodo a la red

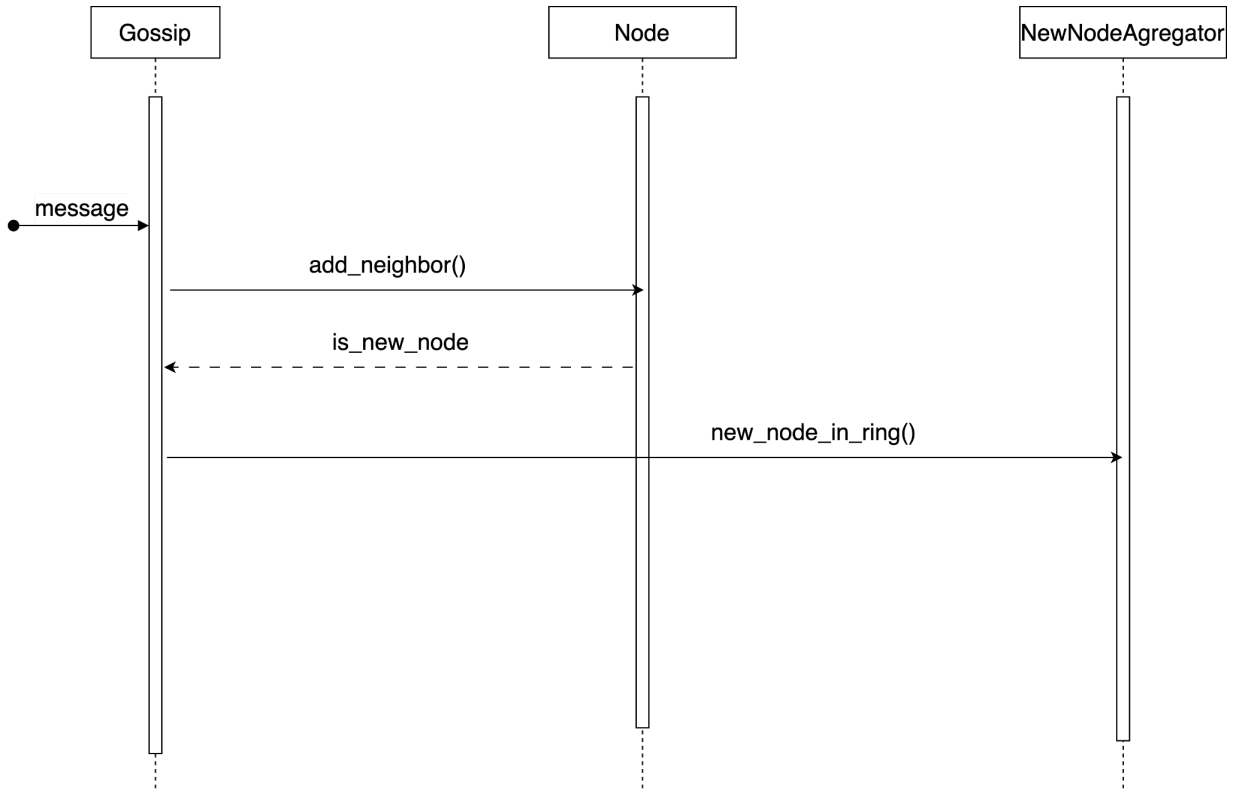
Al haber agregado vnodes por cada nodo físico, cuando se agrega un nuevo nodo, este acepta cantidades aproximadamente iguales de datos de otros nodos del anillo, lo que da como resultado una distribución equitativa de datos en todo el clúster.

Módulos utilizados:



- Identificar que un nodo es nuevo en la red:

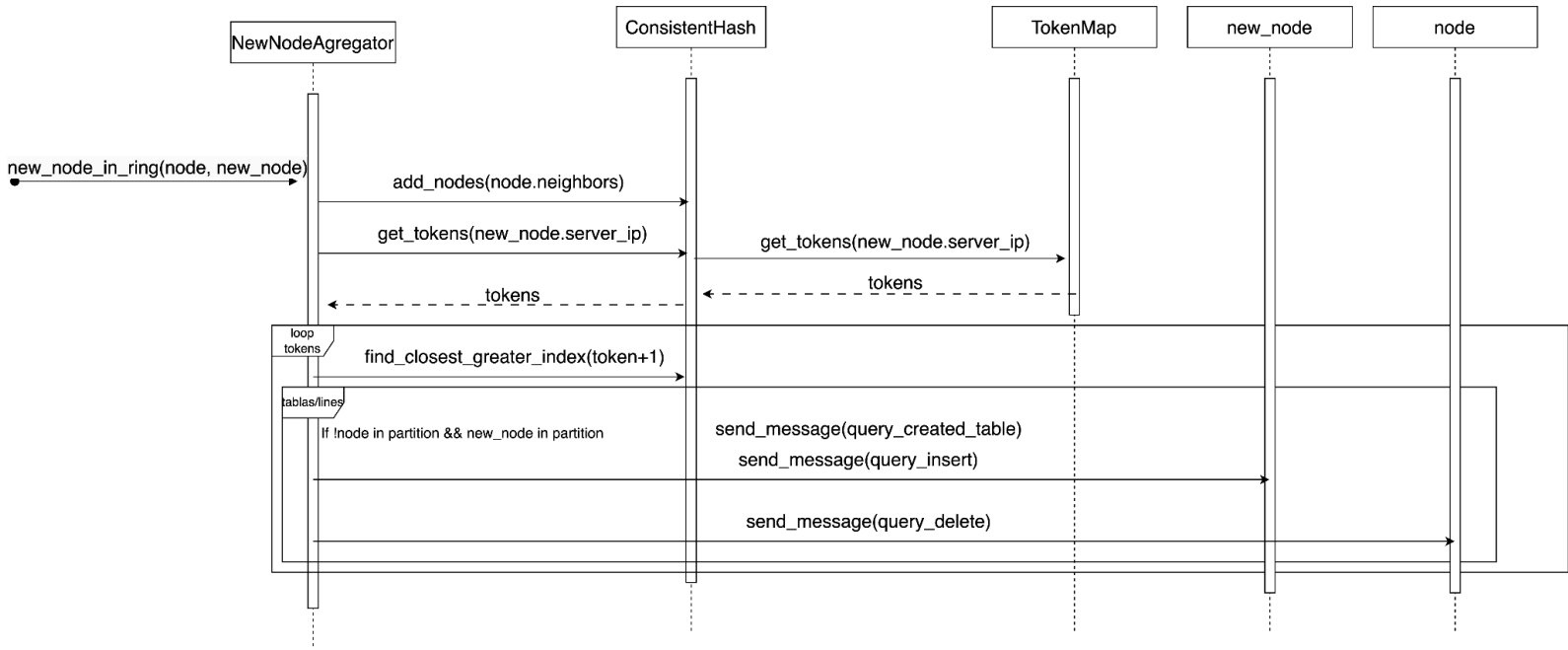
Para la implementación de esta mejora, una de las tareas con las que nos topamos fue identificar cuándo un nodo es nuevo en la red y no es un nodo que se había caído.
Para esto, utilizamos **`Gossip`**. Al recibir un mensaje de tipo **`Gossip`** vamos a identificar si el nodo está en nuestra lista de nodos vecinos, en el caso de no estar consideramos a este un nodo nuevo en la red. Por lo que debemos pasarle la información que le corresponde.



- Nuevas particiones:

Para implementar esto, creamos el módulo **`NewNodeAgregator`** que contiene el **`ConsistentHash`**. Con esté módulo podemos cada nodo que ya estaba conectado en la red e identificó que un nodo es nuevo en la read podrá:

- Enviarle al nuevo nodo las queries de CREATED de las tablas que el nodo ya contenga.
- Enviarle al nuevo nodo la información que tiene que su tabla y ahora no le debería pertenecer más.
- Borrar de su tabla la información que ya no le corresponde de acuerdo a las nuevas particiones.



- Desvinculación de un nodo a la red

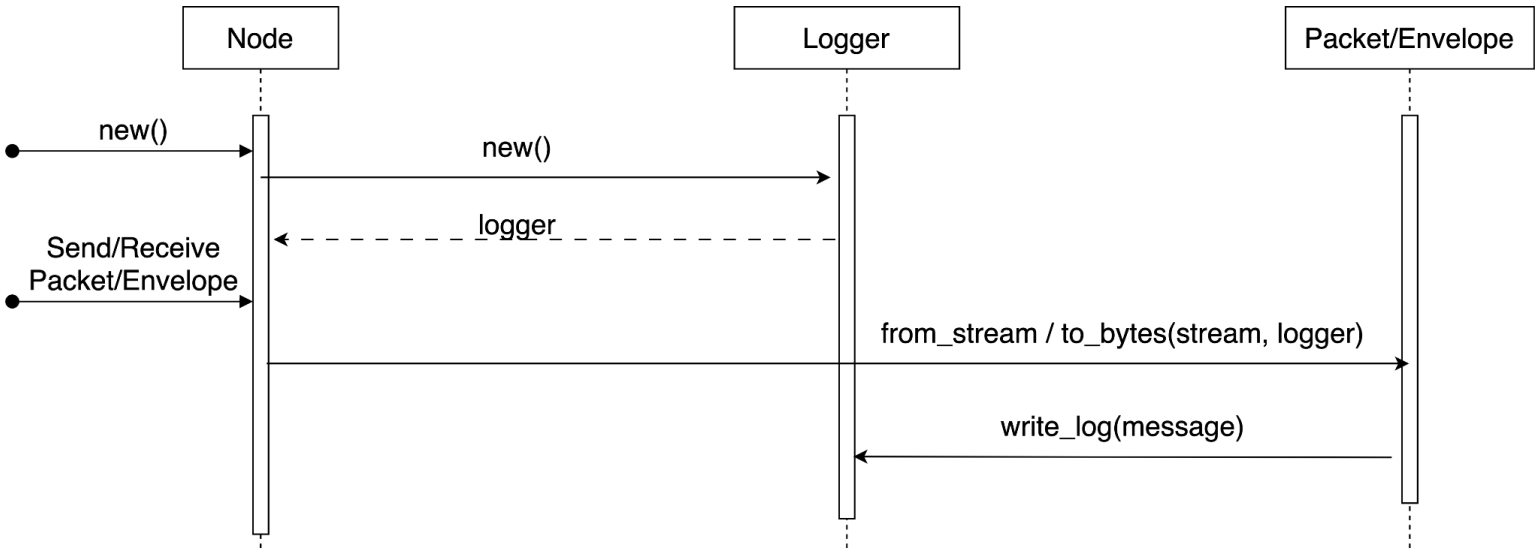
Al caerse un nodo de la red, este permanecerá en el Ring de nodos para darle la posibilidad de volver a conectarse posteriormente y actualizar su información con **`Hinted Handoff`.**
En el caso de que el nodo permanezca caído desde hace un tiempo significativo, se considerará como desvinculado de la red. Y se asignará a un nuevo nodo las particiones que le correspondían. De esta forma nos aseguramos que se mantenga el valor de replicación para todos las líneas.
En cuanto a las líneas que antes poseía el nodo que se desconecto, estas se pasarán al nodo que ocupará su lugar a través de **`Read Repair`**.

4. Logger

Al agregar un logger en el proyecto pudimos tener un resumen de los mensajes que se envían y reciben entre los distintos nodos y al cliente.

Para esta implementación, agregamos el módulo **`Logger`**.
Dicha estructura será creada por cada nodo al conectarse, creando cada uno un archivo de logs.
Dentro del nodo, esta estructura estará protegida por un mutex. De esta manera, cada thread del hilo lockeará la escritura del archivo, de manera que no se sobreescriben y pierdan registros.
Cada vez que se agregue un log en el archivo, se imprimirá también por stdin.

Diagrama de secuencia:



Al conectarse el nodo, se crea el Logger. Y tanto al enviar o recibir un **`Packet`** en la comunicación entre nodos o un **`Envelope`** en la comunicación entre server-client, se pedirá al **`Logger`** que escriba el log en el archivo y en stdin.